

คู่มือเตรียมพรึ่เซนต์ NSC 2026

NUCLEAR Re:Mind — หมวดโปรแกรมเพื่อส่งเสริมทักษะการเรียนรู้ · เฉพาะพรึ่การทำการเกม / โค้ด / สคริปต์
เขียนสำหรับผู้นำเสนอที่ไม่ได้เขียน C# เอง — ทุกหัวข้ออธิบายเป็นภาษาคนก่อน แล้วค่อยแปะศัพท์เทคนิคที่ใช้พูดบนเวที

1. รูปแบบการแข่งขันชนะเลิศ NSC (จากข้อมูลการแข่งขันปีก่อน ๆ)

NSC แข่ง 3 รอบ: รอบข้อเสนอโครงการ → รอบนำเสนอผลงาน → รอบชิงชนะเลิศ โดยรอบชิงจัดเป็นงานนิทรรศการ ผู้เข้าแข่งขัน
ประจำบูธของตัวเอง และมีองค์ประกอบหลัก 4 อย่าง:

องค์ประกอบ	สิ่งที่ต้องเตรียม
บูธ + โปสเตอร์	เกมต้องเปิดเล่นได้ตลอดเวลา (WebGL คือฟอร์แมตที่กรรมการเห็น) · โปสเตอร์สรุประบบเกม 1 แผ่น · เพื่อ Windows build สำรองไว้กรณีเน็ต/เบราว์เซอร์มีปัญหา
สาธิต (Demo)	เตรียมเซฟหลายจุดของเกม: ต้นเกม / ช่วงวิกฤต / ช่วงตอบควิซปลด Mastery — กรรมการมีเวลาน้อย อย่ายให้ต้องเล่นจากวันแรกถึงจะเห็นของดี
นำเสนอ + วิดีโอ	รอบชิงปีก่อน ๆ กำหนดวิดีโอนำเสนอไม่เกิน 7 นาที · พรึ่เซนต์สดต่อกรรมการมักได้ราว 5-10 นาที + ถามตอบ
สัมภาษณ์	กรรมการเดินเข้าบูธถามเจาะทีละคนได้ — ทุกคนในทีมต้องอธิบายภาพรวมได้ ไม่ใช่เฉพาะคนเขียนโค้ด (นี่คือเหตุผลของเอกสารฉบับนี้)

เกณฑ์ที่กรรมการใช้ (สรุปจากเกณฑ์ NSC ปีก่อน ๆ): [ความสมบูรณ์ของผลงาน](#) [เทคนิคที่ใช้](#) [ความคิดสร้างสรรค์](#)
[Look & Feel / ใช้งานง่าย](#) [การนำเสนอ+ตอบคำถาม](#)

⚠ สำคัญ

กติกา/กำหนดการของ NSC2026 รอบชิงจริงต้องยึดประกาศจากระบบ GENA ของ NSC เป็นหลัก — ตัวเลขในตารางนี้มาจากปีก่อน ๆ ใช้เตรียมตัวได้แต่ต้องเช็คของจริงอีกครั้ง

2. เกมนี้สร้างด้วยอะไร — พูดให้ถูกใน 30 วินาที

✓ บทพูดเปิด (ท่องได้เลย)

"เกมพัฒนาด้วย **Unity 6** เขียนด้วยภาษา **C#** ใช้ระบบเรนเดอร์ **URP 2D** ที่มีแสงไฟ 2 มิติจริง ๆ ในฉาก ตัวเกมส่งออกเป็น **WebGL** เล่นบนเบราว์เซอร์ได้เลยไม่ต้องติดตั้ง โค้ดทั้งหมดประมาณ **180+ ไฟล์** ออกแบบเป็นระบบแยกส่วนที่คุยกันผ่านศูนย์กลางข้อความตัวเอง ทำให้เพิ่มระบบใหม่ได้โดยไม่พังของเก่า"

ศัพท์	ความหมายภาษาคน
Unity	"โปรแกรมสร้างเกม" (เกมเอนจิน) — จัดการภาพ เสียง ฟิสิกส์ ให้เราเขียนแค่กติกาเกม
C# (ซี-ชาร์ป)	ภาษาโปรแกรมที่ Unity ใช้ ไฟล์นามสกุล <code>.cs</code> เรียกว่า "สคริปต์" 1 ไฟล์ $\approx$ 1 ระบบย่อย
Scene (ฉาก)	ไฟล์ห้อง 1 ห้องของเกม — เกมนี้มี 2 ฉากหลัก: เมนู (MainMenu) กับตัวเกม (GameScene)
URP 2D + Light 2D	ระบบวาดภาพของ Unity ที่ให้หลอดไฟ 2 มิติจริง — ตึกมีแสงหน้าต่าง ไฟดับได้ตอนไฟไม่พอ
WebGL	ฟอร์แมต "เกมบนเบราว์เซอร์" — แปลงโค้ด C# ทั้งเกมเป็นภาษาที่เบราว์เซอร์รันได้ (WebAssembly)
ScriptableObject	"แฟ้มข้อมูล" ของ Unity — เนื้อหาเกม (การ์ด, ครีษ, ตึก) เก็บเป็นไฟล์ข้อมูลแยกจากโค้ด แก่เนื้อหาได้โดยไม่แตะโค้ดเลย

3. สถาปัตยกรรมโค้ด — แผนผังที่ต้องเล่าได้

3.1 หลักการใหญ่: ทุกระบบคุยกันผ่าน "เสียงตามสาย" ตัวเดียว

โค้ดเกมนี้แบ่งเป็น ระบบย่อย ~15 ระบบ (คนงาน, ความหวัง, วิจัย, การวิจัย, พายุ ฯลฯ) กติกาหลักคือห้ามระบบหนึ่งไปสั่งอีกระบบตรง ๆ — ทุกอย่างต้องประกาศผ่านตัวกลางชื่อ `EventManager`

อุปมาที่ใช้พูดได้

EventManager เหมือนเสียงตามสายของโรงเรียน: ฝ่ายครัวไม่เดินไปบอกครูที่ละห้องว่า "ข้าวหมดแล้ว" แต่ประกาศกลาง 1 ครั้ง ("ResourceDepleted: Food") แล้วห้องไหนสนใจก็รีบไปทำต่อเอง — ระบบคนงานได้ยินก็ทำให้คนหิว ระบบความหวังได้ยินก็หักคะแนน ระบบการ์ดได้ยินก็เชื่อว่าถึงเงื่อนไขวิกฤต "ความอดอยาก" หรือยัง

ข้อดีที่กรรมการฟังแล้วพยักหน้า: เพิ่มระบบใหม่ = เพิ่มผู้ฟังคนใหม่ ไม่ต้องแก้โค้ดคนพูดสักบรรทัด ทำให้ทีมพัฒนา 6 สปรินต์ต่อเนื่องโดยระบบเก่าไม่พัง

3.2 โครงสร้างโฟลเดอร์โค้ด (Assets/Scripts/)

โฟลเดอร์	ทำหน้าที่	ไฟล์เด่นที่ควรจำชื่อ
Core/	หัวใจเกม: นาฬิกาวัน, เต่าปฏิกรณ์, ทรัพยากร, ค่ากลาง ทั้งเกม	GameManager, TimeManager, ReactorController, GameConfigSO
Workers/	คนงาน 14 คน: สุขภาพ ความล่า ประสิทธิภาพ ภาระงาน	WorkerManager, ShiftSystem
Hope/	ค่าความหวังของเมือง — แพ้ถ้าเป็น 0	HopeLedger (บัญชีรายรับ-รายจ่าย)
Research/	ห้องวิจัย: คิววิจัย, โหนดความรู้ 8 ใบ	ResearchLab, KnowledgeDB, NoteCardPopup
Mastery/	ความเชี่ยวชาญถาวรจากการตอบควิซถูก	MasteryRegistry (ระบบอินมา "ถาม" ตัวนี้)
Cards/	การ์ดวิฤต 8 ใบ + ตัวเลือกที่ล็อกด้วยความรู้	CardManager, CrisisCardSO
Codex/	สารานุกรมในเกม 11 หัวข้อ + ควิซ 11 ข้อ	CodexQuizManager
Storm/ ZoneB/ Barks/ Records/ Inventory/	พายุรังสี · เขตอันตราย · บทพูด NPC · คุ้ข้อมูล · คลังไอ เทม	StormSystem, ZoneBController, BarkManager, DataRecovery, InventoryManager

4. กลไกเด่นเชิงเทคนิค — จุดขายที่ต้องพูดให้ครบ

4.1 "ความรู้ไม่ใช่รางวัล — มันคือกลไกที่ทำให้ชนะ" (จุดขายอันดับ 1)

✓ บทพูด

"เกมการศึกษาส่วนใหญ่ให้ตอบควิซแล้วได้แต้ม/ดาว แต่เกมเราผูกความรู้เข้ากับตัวเลขจริงของระบบเกม เช่น ถ้าผู้เล่นเข้าใจเรื่อง Tritium Breeding แล้วตอบควิซถูก อัตราผลิตเชื้อเพลิงของ Zone B จะเพิ่มจาก 3.0 → 8.0 ต่อวัน — เราพิสูจน์ด้วยการรันจำลองว่าถ้าไม่ตอบควิซเลย จะแพ้ 100% ทุกแนวการเล่น ความรู้จึงเป็นเงื่อนไขการชนะจริง ๆ ไม่ใช่ของแถม"

เบื้องหลังโค้ด: ตอบควิซถูก → `MasteryRegistry` บันทึกการ → ระบบต่าง ๆ (เตาปฏิกรณ์, ฟาร์ม, โรงพยาบาล, Zone B) มาถามตัวนี้ทุกวันว่า "ผู้เล่นรู้เรื่อง X หรือยัง" แล้วปรับตัวเลขผลผลิตให้ทันที — จุดเดียว เก็บถาวร ใช้ข้ามระบบ

4.2 วิฤตเกิดจาก "สภาพเมือง" ไม่ใช่ "ปฏิทิน" (กติกาหลักของทีม)

✓ บทพูด

"เรามีกติกาห้ามเขียนโค้ดแบบ 'พอถึงวันที่ 12 ให้เกิดเหตุการณ์ A' — ทุกวิฤตผูกกับตัวชี้วัดสถานะจริง เช่น การ์ดความร้อนเกิดเมื่อ $HEAT \geq 62$ การ์ดโรคระบาดเกิดเมื่อคนป่วยเกิน 3 คน ทำให้ผู้เล่น 2 คนเล่นเกมเดียวกันแล้วเจอเหตุการณ์คนละแบบตามการตัดสินใจของตัวเอง — replay ได้จริง และผู้เล่นเรียนรู้ว่าการละเลยระบบไหนพาไปสู่วิฤตแบบไหน"

เบื้องหลังโค้ด: ทุกสิ้นวัน `CardManager` ถ่ายภาพสถานะเมือง (ความร้อน, คนหิว, คนป่วย, รังสี...) แล้วไล่เช็คกับเงื่อนไขของการ์ดทั้ง 8 ใบ ไปไหนเข้าเงื่อนไขและผ่านระยะคุลดาวน → แต่งเป็นบทสนทนาตัวละคร แล้วให้ผู้เล่นเลือกทางออก ซึ่งทุกทางเลือกมีราคา ไม่มีทางเลือกฟรี

4.3 ตัวเลือกที่ลือด้วยความรู้ — "โชว์ให้เห็นแต่กดไม่ได้"

ในการ์ดวิฤต ตัวเลือกที่ดีที่สุดจะลืออยู่จนกว่าจะวิจัยเรื่องนั้นสำเร็จ และเราจงใจแสดงให้เห็นพร้อมบอกว่าต้องวิจัยอะไร (ไม่ซ่อน) — ผู้เล่นเห็นชัดว่า "ถ้าเรารู้มากกว่านี้ เราจะมีทางเลือกที่ดีกว่านี้" นี่คือการสอนคุณค่าของความรู้ผ่านดีไซน์ ไม่ใช่ผ่านคำเทศน์

4.4 สมดุลเกมด้วยการจำลอง (Simulation-driven balancing)

✓ บทพูด

"ค่าตัวเลขในเกมไม่ได้เดาจากความรู้สึก — เรามีสคริปต์จำลองการเล่น (Python) รันเกมจบ 30 วันแบบอัตโนมัติหลายพันรอบด้วยแนวการเล่นต่างกัน แล้วดูสถิติแพ้-ชนะ ตัวอย่างที่เจอจริง: ค่าอัตราเพิ่ม CORE ตอนบูสต์ ถาลดจาก 3.0 เหลือ 2.3 ผู้เล่นจะแพ้ 100% ทันทีทุกแนว — ค่านี้เลยถูกล็อกห้ามแก้ และตัวเลขทุกตัวถูกรวมไว้ในไฟล์ตั้งค่ากลางไฟล์เดียว (GameConfigSO) แก่ทีเดียว มีผลทั้งเกม ไม่มีเลขฝั่งกระจายโน้ต"

การจำลองยังจับบั๊กดีไซน์ได้ก่อนเขียนเกมจริง เช่น สูตรระบายความร้อนที่ไม่มีเพดานทำให้เตาไร้ความหมาย, การหักค่าวิจัยรายวันที่ทำให้วิจัยได้ใบเดียวแล้วตัน — มีบันทึกไว้ในเอกสารออกแบบมากกว่า 15 รายการ

4.5 ระบบคนงานคิดที่ "ประสิทธิภาพ" ไม่ใช่ "จำนวนหัว"

ผลผลิตของตึก = ผลรวมประสิทธิภาพรายคน (Σ efficiency) ไม่ใช่นับจำนวนคน — คนหิว/ป่วย/ล้า ประสิทธิภาพตก ผลผลิตตกจริง ผู้เล่นจึงต้องดูแลคุณภาพชีวิตคนงาน ไม่ใช่แค่ยัดคนลงตำแหน่ง และค่าความหวังของเมืองก็ห้ามเขียนตรง ๆ — ทุกระบบส่ง "รายการบัญชี" เข้า HopeLedger ผู้เล่นเปิดดูได้ว่าวันนี้ +เท่าไร -เท่าไร จากอะไร โปรงใส่ทุกแต้ม

4.6 เนื้อหาเกมแยกจากโค้ด 100% (Data-driven content)

การ์ดวิฤต 8 ใบ · คริซ 11 ข้อ · โน้ตวิจัย 8 ใบ · Codex 11 หัวข้อ · บทพูด 62 บรรทัด — ทั้งหมดเป็น **ScriptableObject** (แฟ้มข้อมูล) นักออกแบบ/นักเขียนแก้เนื้อหา เพิ่มการ์ด เปลี่ยนคำถามได้โดยไม่แตะโค้ดแม้แต่บรรทัดเดียว — นี่คือการตอบเวลากรรมการถามว่า "จะขยายเนื้อหาไปวิชาอื่นได้ไหม"

5. คำถามที่คาดหวังกรรมการจะถาม + แนวตอบ

เรียงจากโอกาสเจอบ่อย → น้อย · แนวตอบเขียนเป็นภาษาพูด ปรับเป็นสำนวนตัวเองได้ · กติกาสำคัญ: **ตอบตามบทบาทจริงของเราในทีม อย่าเคลมส่วนที่คนอื่นทำ** — กรรมการ NSC เจอผู้แข่งมาหลายร้อยทีม จับได้ทันทีถ้าตอบเกินความรู้จริง และการอธิบาย "สถาปัตยกรรม+เหตุการณ์การออกแบบ" ได้ลึก น่าประทับใจกว่าแค่ลงอธิบาย syntax แล้วพลาต

5.1 กลุ่มคำถามเรื่องความเป็นเจ้าของผลงาน (เจอแน่นอน)

Q: "ใครเขียนโค้ดส่วนไหนบ้าง / น้องเขียนเองทั้งหมดหรือเปล่า"

แนวตอบ: แบ่งบทบาทตามจริง เช่น "ผมรับผิดชอบเกมดีไซน์ทั้งหมด — ออกแบบระบบ กติกา สมดุลเกม เนื้อหาการเรียนรู้ และกำกับการพัฒนา ส่วนการเขียน C# ใช้ [เครื่องมือ/ผู้ช่วยตามจริงของทีม] โดยผมเป็นคนกำหนดสเปกทุกระบบ ตรวจสอบ และทดสอบทุกฟีเจอร์" — จากนั้น**รับต่อด้วยความลึกที่เราจริงจัง:** อธิบายเหตุผลดีไซน์ข้อ 4.1–4.4 ได้คล่อง = พิสูจน์ว่าเราเข้าใจผลงานตัวเองจริง

Q: "ใช้ AI ช่วยพัฒนาหรือเปล่า ใช้ยังไง"

แนวตอบ (ถ้าใช้จริง — ตอบตรง ๆ ดีกว่าปฏิเสธ): "ใช้ครับ เราใช้ AI เป็นเครื่องมือเร่งงานเหมือนที่โปรแกรมเมอร์อาชีพใช้กัน แต่หัวใจของผลงาน — แนวคิด 'ความรู้คือกลไกชนะ', ดีไซน์ระบบทั้งหมด, เนื้อหาฟิสิกส์นิวเคลียร์, การตรวจสอบสมดุลด้วย simulation — มาจากทีมเรา AI เขียนโค้ดตามสเปกที่เรากำหนดและเราเป็นคนทดสอบ/ตัดสินใจทุกอย่าง" · เตรียมตัวอย่าง 1 เรื่องที่ "เราสั่งแก้ AI" เพื่อโชว์ว่าเราคุมมัน ไม่ใช่มันคุมเรา

Q: "ถ้าให้แก้ฟีเจอร์ X ตรงนี้เดี๋ยวนี้ ทำได้ไหม"

แนวตอบ: ชี้ไฟล์ถูกจุด = ผ่าน "ค่าเกมทุกตัวอยู่ใน GameConfigSO แก้ที่เดียวมีผลทั้งเกมครับ เช่นถ้าจะให้คนงานโตเร็วขึ้นผมแก้ growthChance ในไฟล์นี้" — **ไม่ต้องเขียนโค้ดสด แค่รู้ว่าอะไรอยู่ไหนเพราะอะไร**

5.2 กลุ่มคำถามเชิงเทคนิค

Q: "ทำไมเลือก Unity / ทำไมไม่ใช้เอนจินอื่น"

"สามเหตุผลครับ: 1) Unity ส่งออก **WebGL** ได้ตรง ๆ — กรรมการและผู้ใช้เปิดเล่นบนเบราว์เซอร์ได้เลยไม่ต้องติดตั้ง เหมาะกับโปรแกรมเพื่อการเรียนรู้ที่ต้องเข้าถึงง่าย 2) URP 2D ให้แสงไฟ 2 มิติที่สร้างบรรยากาศเมืองหลังหายนะได้จริง 3) ระบบ ScriptableObject ทำให้แยกเนื้อหาการเรียนรู้ออกจากโค้ด ขยายเนื้อหาได้โดยไม่ต้องเขียนโปรแกรมเพิ่ม"

Q: "ระบบไหนซับซ้อนที่สุด / ภูมิใจอะไรที่สุดในโค้ด"

แนะนำตอบ **MasteryRegistry + วงจรความรู้:** "ระบบที่ภูมิใจสุดคือวงจร 'เรียน-ใช้-ทดสอบ-ได้ถาวร' ครับ: ผู้เล่นวิจัยความรู้ → ต้อง**ใช้ความรู้**นั้นจริงในเกมก่อน (เช่นเดินเครื่องสกัตครั้งแรก) คริซถึงจะปลดให้ตอบ → ตอบถูกได้ Mastery ถาวรที่ 6 ระบบทั่วเกมมาอ่านค่าไปเพิ่มผลผลิตจริง — โค้ดต้องประสาน 4 ระบบ (วิจัย/คริซ/ทะเบียนความเชี่ยวชาญ/ระบบผลิต) โดยไม่พันกัน ซึ่งทำได้เพราะสถาปัตยกรรม event-driven"

Q: "เกมบาลานซ์ยังไง รู้ได้ไงว่าไม่ยาก/ง่ายเกินไป"

ยกข้อ 4.4 เติม ๆ — มี simulation รันพันรอบ + ตัวเลขล็อกที่พิสูจน์แล้วว่าแตะแล้วพัง + บั๊กดีไซน์ 15+ รายการที่ sim จับได้ก่อนถึงมือผู้เล่น นี่คือการตอบที่ทีมส่วนใหญ่ไม่มี

Q: "เซฟเกมทำงานยังไง / ปิดเบราว์เซอร์แล้วเซฟหายไหม"

"เซฟเป็นไฟล์ JSON ครับ บน Windows เก็บในเครื่อง บน WebGL เก็บใน **IndexedDB** ของเบราว์เซอร์ — ปิดแท็บแล้วกลับมาเล่นต่อได้ และเราออกแบบให้เซฟเก่าเปิดกับเกมเวอร์ชันใหม่ได้เสมอ (ทุกฟีลด์ใหม่มีค่าเริ่มต้น เซฟเก่าไม่พัง)"

Q: "ทำไมบางจังหวะบน WebGL ช้า/สะดุด"

"WebGL รันโค้ดผ่าน WebAssembly ในเบราว์เซอร์ครับ ประสิทธิภาพต่ำกว่ารันตรงบน Windowsราว 20–30% เราจึงลดงานต่อเฟรม เช่น แคชการค้นหาวัดถุไว้ตั้งแต่เริ่มฉากแทนการค้นหาค้นทุกเฟรม และตัวเกมเป็นแนววางแผนรายวัน ไม่ใช่เกมแอ็กชัน จึงไม่กระทบการเล่น"

5.3 กลุ่มคำถามด้านการเรียนรู้ (หมวดนี้โดนถามหนักสุด)

Q: "วัดได้ยังไงว่าผู้เล่นได้เรียนรู้จริง ไม่ใช่แค่สนุก"

"ตัวเกมวัดในตัวเองครับ: ควิช 11 ข้อคือแบบทดสอบที่แฝงเป็นกลไกเกม — สถิติการตอบ (ถูก/ผิด/ข้อไหน) เก็บอยู่ในระบบ และผลการเล่นสะท้อนความเข้าใจโดยตรง เพราะเราพิสูจน์แล้วว่าผู้เล่นที่ไม่เข้าใจเนื้อหาจะชนะเกมไม่ได้ · ส่วนการวัดภายนอกแนวทางต่อยอดคือ pre-test/post-test กับกลุ่มผู้เรียนจริง" (ถ้าเคยให้เพื่อน/รุ่นน้องทดลองเล่น — เล่าตรงนี้ พร้อมสิ่งที่แก้จาก feedback)

Q: "เนื้อหาฟิสิกส์ถูกต้องไหม ใครตรวจ"

เตรียมแหล่งอ้างอิงเนื้อหานิวเคลียร์/ฟิสิกส์ที่ใช้เขียน Codex และชื่อครูที่ปรึกษาที่ช่วยตรวจ — ควรตอบได้ว่า Deuterium สกัตกจากน้ำจริง (1 ใน ~6,400 อะตอมไฮโดรเจน) และ Tritium breeding คืออะไร เพราะสองเรื่องนี้เป็นกลไกหลักของเกม

Q: "ต่างจากเกม/สื่อการสอนอื่นที่มีอยู่แล้วยังไง"

"สื่อส่วนใหญ่ให้ความรู้เป็นรางวัล—ตอบถูกได้ดาว เกมเราให้ความรู้เป็นอำนาจ—รู้แล้วปลดทางเลือกใหม่ เพิ่มผลผลิตจริง ชนะได้จริง ผู้เล่นจึงอยากรู้เพราะต้องใช้ ไม่ใช่เพราะถูกสั่งให้จำ และวิกฤตที่เกิดจากการตัดสินใจของผู้เล่นเองทำให้บทเรียน (เช่น ผลของรังสี, การจัดการพลังงาน) เป็นประสบการณ์ตรง ไม่ใช่ข้อความในตำรา"

Q: "ถ้าตอบควิชผิด เกมลงโทษไหม เด็กจะเครียดไหม"

"ไม่ลงโทษครับ — ตอบผิดเกมเฉลยพร้อมคำอธิบายเสมอ แล้วให้กลับมาตอบใหม่ได้วันถัดไป ควิชทั้งหมดข้ามได้ ไม่บังคับตอบ เราจึงออกแบบให้ 'ความรู้คือโอกาส ไม่ใช่โทษลงโทษ' สอดคล้องหลักการเรียนรู้ที่ผิดพลาดได้อย่างปลอดภัย"

5.4 กลุ่มคำถามต่อยอด

Q: "จะพัฒนาต่อยังไง / ขยายไปวิชาอื่นได้ไหม"

"ได้ครับ เพราะเนื้อหาแยกจากโค้ดทั้งหมด (ข้อ 4.6) — เปลี่ยนชุดการ์ด/ควิช/Codex เป็นเรื่องพลังงานแสงอาทิตย์ ชีววิทยา หรือ เศรษฐศาสตร์ ได้โดยใช้เอนจินเกมเดิมทั้งตัว · แผนถัดไป: [เลือกตามจริง เช่น เพิ่มระดับความยาก / โหมดครูดสถิติการตอบของนักเรียน / แปลอังกฤษ]"

Q: "ทำกันกี่คน นานเท่าไร อะไรยากที่สุดในการพัฒนา"

ตอบตามจริง + ปิดด้วยบทเรียน 1 ข้อ เช่น "ยากสุดคือการทำให้ 15 ระบบทำงานพร้อมกันโดยไม่พึ่งกันเอง ซึ่งแก้ด้วยกติกากา event-driven กับการทดสอบอัตโนมัติ 500+ ข้อที่รันทุกครั้งก่อนรวมโค้ด"

6. เดโมสคริปต์ 5 นาทีหน้าบูธ (เดินตามนี้)

นาที	ทำอะไรบนจอ	พูดอะไร
0:00–0:30	หน้าเมนู → เข้าเกม	เปิดตัว 1 ประโยค: "ผู้เล่นคือ Dr. Auren มีเวลา 30 วันดัน CORE TOWER ถึง 100% ก่อนพายุรังสีมา — และทางเดียวที่จะชนะคือ เข้าใจฟิสิกส์นาเคลียร์ จริง ๆ"
0:30–1:30	ซื้อ HUD: ทรัพยากร 6 ชนิด, Hope, คนงาน	เล่าอุป 1 วัน: จัดคนเข้าตึก → ตีผลลัพธ์ตามประสิทธิภาพคน → สิ้นวันเช็ควิกฤต — ชี้ว่าคนหิวประสิทธิภาพตกจริง (ข้อ 4.5)
1:30–2:30	เปิดห้องวิจัย → เลือกโน้ต → โชว์ NoteCard ความรู้	"วิจัยเสร็จได้ ความรู้จริง ไม่ใช่แค่ตัวเลข — เช่นตัวเทอเริ่มสกัดจากน้ำ แล้วเกมก็ปลดปุ่มสกัดจริง ๆ ให้ใช้ความรู้นั้น"
2:30–3:30	เซฟที่เตรียมไว้: การ์ดวิกฤตแดง → ซื้อตัวเลือกที่ล็อก 🔒	จุดพีคของเดโม: "ตัวเลือกที่ดีที่สุดเห็นแต่ กดไม่ได้ จนกว่าจะวิจัย — ความรู้ = อำนาจต่อตรงกับวิกฤต"
3:30–4:30	เปิด Codex → ตอบควิซ 1 ข้อ (เตรียมข้อที่มั่นใจ) → โชว์ Mastery + ผลผลิตเพิ่ม	"ตอบถูกครั้งเดียว ระบบผลิตทั้งเกมเปลี่ยนถาวร — และถ้าไม่ตอบเลย เราพิสูจน์ด้วย simulation ว่าแพ่ 100%"
4:30–5:00	กลับภาพรวมเมือง (มีแสงไฟ 2D สวย ๆ)	ปิด: เทคโนโลยี (Unity/C#/WebGL/180+ ไฟล์/event-driven) + เชิญกรรมการลองเล่นเอง

⚠️ เตรียมก่อนวันจริง

- **เซฟ 3 จุด:** ตันเกม (สอนอุป) · กลางเกมมีวิกฤตพร้อมแดง · ช่วงควิซพร้อมตอบ — ซ้อมเรียกทั้ง 3 จุดให้คล่อง
- **แผนสำรอง:** Windows build + วิดีโอเกมเพลย์ในเครื่อง เพื่อ WebGL/เน็ตงานมีปัญหา
- **ซ้อมตอบ 5.1 ให้สั้นที่สุด** — คำถามเรื่องบทบาทในทีมคือด้านแรกและด้านสำคัญที่สุดของหมวดนี้
- ทดลองให้คนไม่เคยเล่น เล่น 10 นาทีโดยเราห้ามพูด — จุดที่เขางง คือจุดที่กรรมการจะงง

7. พจนานุกรมฉุกเฉิน — ศัพท์ที่อาจหลุดจากปากกรรมการ

ถ้ากรรมการพูดว่า...	เขาหมายถึง
Architecture / สถาปัตยกรรม	ภาพรวมว่าโค้ดแบ่งส่วนยังไง คุยกันยังไง → ตอบด้วยข้อ 3 (event-driven + 15 ระบบ)
Game loop	วงจรหลักที่เกมวนทำซ้ำ → ของเรา: จัดคน→ผลิต→สิ้นวัน→เช็ควิกฤต→วันใหม่
Performance / Optimize	เกมเล่นไหม กินเครื่องแค่ไหน → ตอบด้วยแนวข้อ 5.2 (แคช, ลดงานต่อเฟรม)
Scalability / ขยายระบบ	เพิ่มเนื้อหา/ฟีเจอร์ได้ง่ายไหม → ตอบด้วยข้อ 4.6 (data-driven)
Version control / Git	ระบบเก็บประวัติโค้ด ย้อนกลับได้ ทำงานเป็นทีมได้ — โปรเจกต์นี้ใช้ Git มีประวัติ commit ต่อเนื่องทุกฟีเจอร์
Unit test / การทดสอบ	โค้ดทดสอบโค้ด — โปรเจกต์นี้มีชุดทดสอบอัตโนมัติ 500+ ข้อ (EditMode tests) ครอบคลุมสูตรคำนวณสำคัญ รันก่อนรวมโค้ดทุกครั้ง
Asset	ไฟล์ทรัพยากรเกม: ภาพ เสียง ฟอนต์ ข้อมูล — ภาพเกมนี้อีสไต้ pixel-art isometric
Build / Deploy	การแปลงโปรเจกต์เป็นตัวเกมจริง → เรามีทั้ง WebGL (ส่งกรรมการ) และ Windows

แหล่งข้อมูลรูปแบบการแข่ง: เว็บไซต์ NECTEC/NSTDA (โครงการ NSC), scimath.org (บทความ NSC), คู่มือการแข่งขัน NSC ปีก่อน ๆ และบันทึกประสบการณ์ผู้เข้าแข่งขันรอบชิง · กติกา NSC2026 จรียึดประกาศทางการจากระบบ GENA · ข้อมูลเทคนิคเกมทั้งหมดอ้างอิงจากโค้ดจริงของโปรเจกต์ ณ ก.ค. 2026